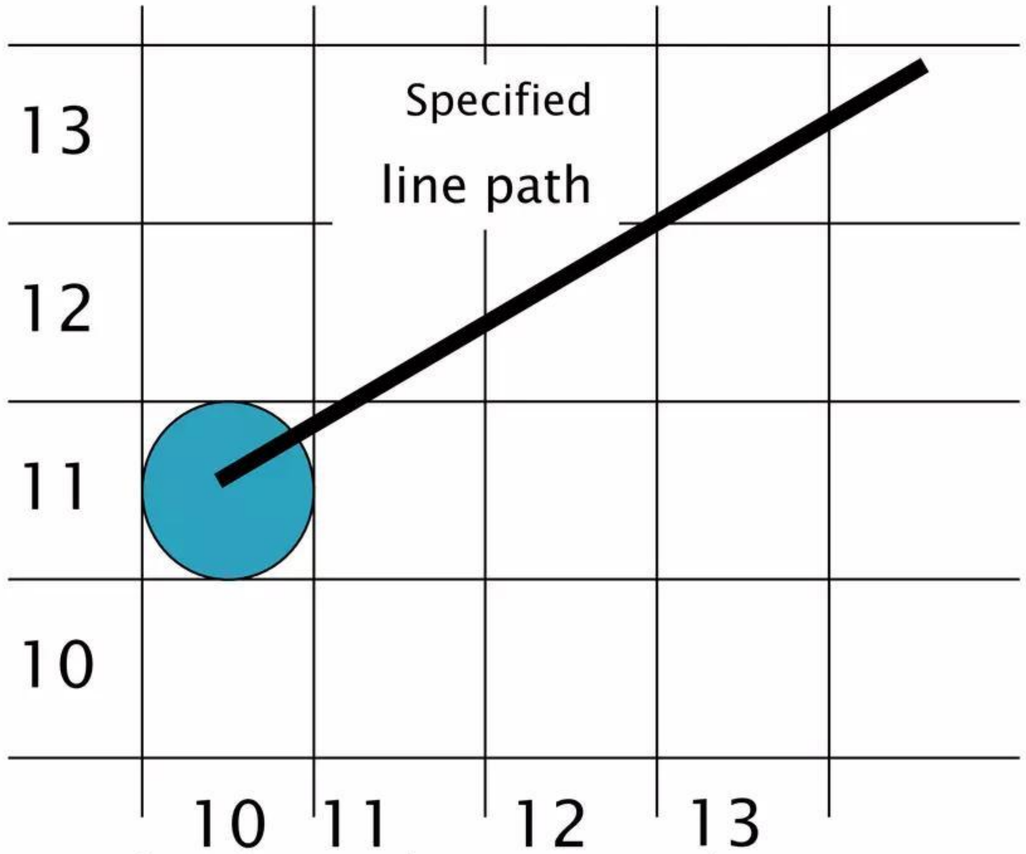
Bresenham's line drawing algorithm

Prof. Neeraj Bhargava
Pooja Dixit
Department of Computer Science
School of Engineering & System Sciences
MDS, University Ajmer, Rajasthan, India

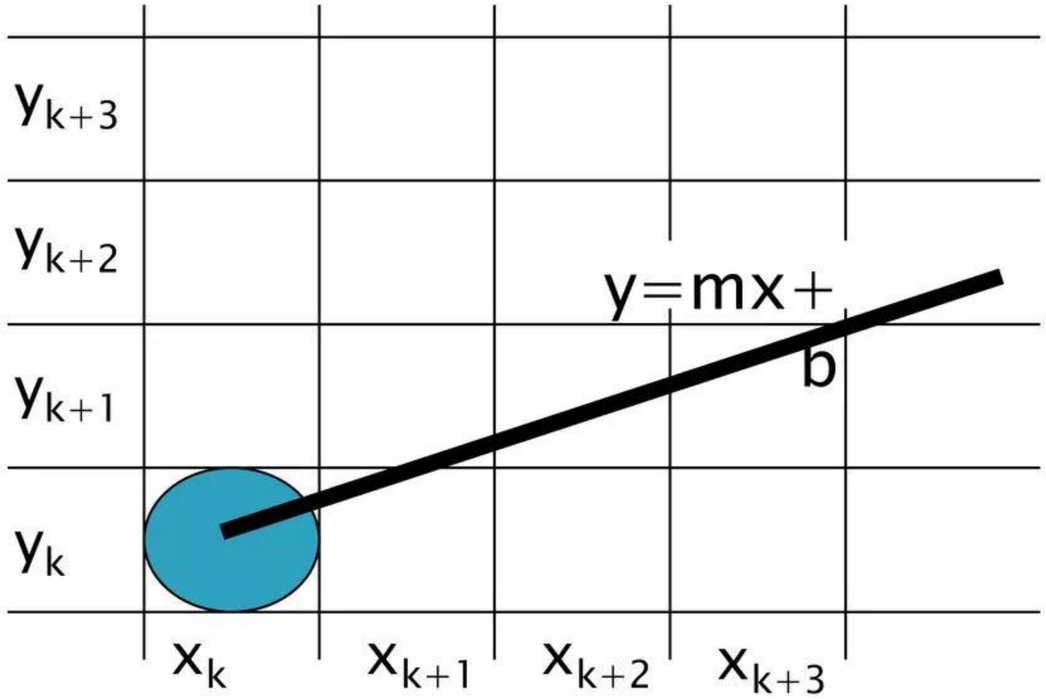
Bresenham's line algorithm

- Bresenham Line drawing algorithm is used to determine closest points to be illuminated on the screen to form a line.
- As we know a line is made by joining 2 points, but in a computer screen, a line is drawn by illuminating the pixels on the screen.



Decide what is the next pixel position:
(11,11) or (11,12)

Illustrating Bresenham's Approach



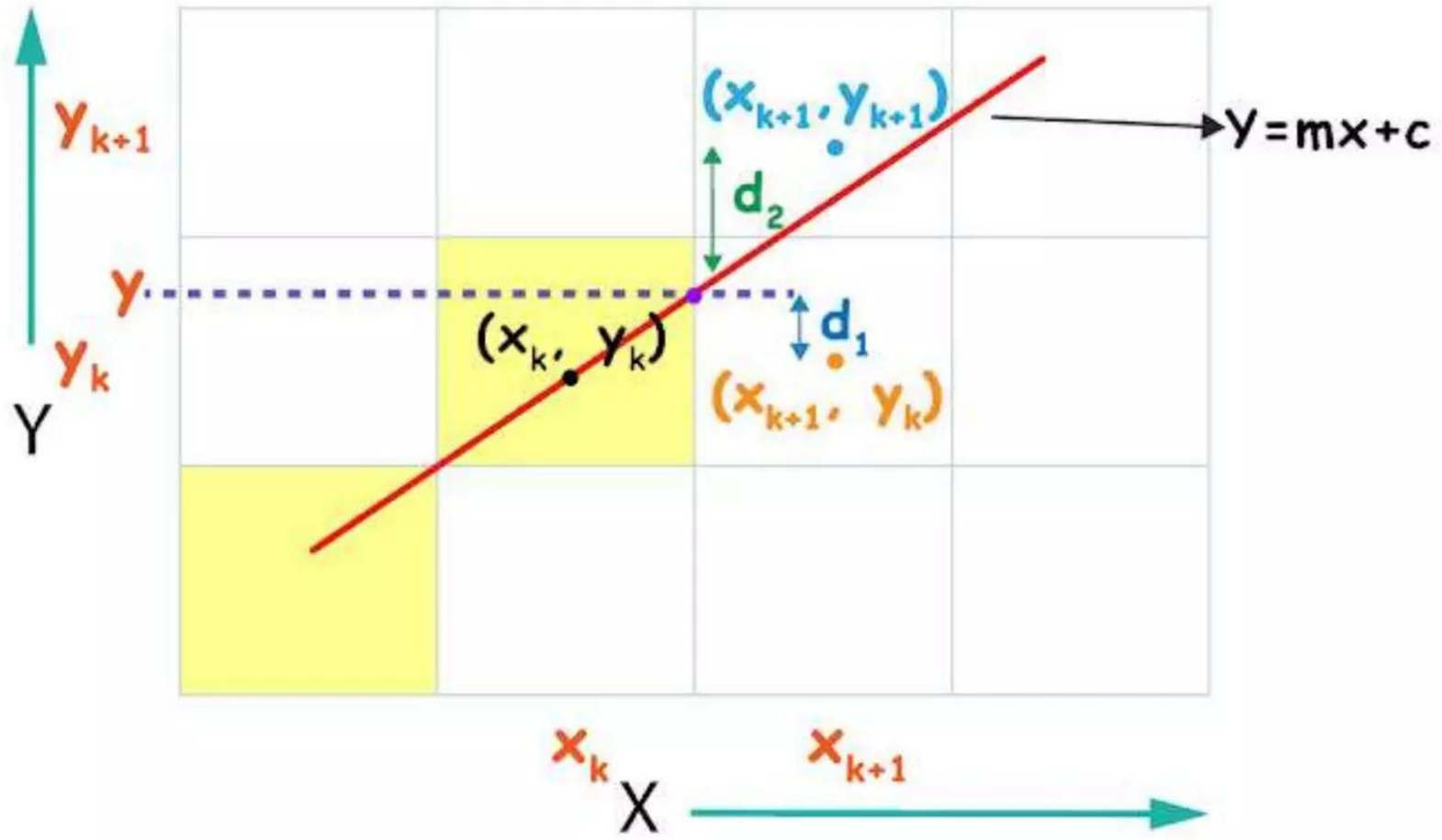
- For the pixel position $x_{k+1}=x_k+1$, which one we should choose:
- (x_{k+1},y_k) or (x_{k+1}, y_{k+1})

➢The slope of a line plays a major role in the line equation that's why Bresenham line drawing algorithm calculates the equation according to the slope of the line.

➢The slope of the line can be greater than 1 (m>1) or less than or equal to 1 (m<=1).

Now enough talking let's derive the equation.

Derivation:



- To draw a line we have to calculate the points or pixels to be illuminated on the screen.

  Now while drawing a line a sometimes it passes through 2 pixels at the same time then we have to choose 1 pixel from the 2 to illuminate it.

  so, the bresenham algorithm calculates the distance from the intersection point y to both the pixels and whichever is smaller we choose that pixel.

Derivation

- The equation of Line is:
- $Y=mx+c$ ....(1)
- Here m is the slope of Line and C is y-Intercept of line.
  The slope can also be written as

$$\frac{y_2 - y_1}{x_2 - x_1} \text{ or } \frac{\text{change in } x}{\text{change in } y} \text{ or } \frac{dx}{dy} \qquad .....(2)$$

- First we calculate the slope of the line if slope is less than 1, then X will always be incremented.
  so at ($m<=1$)
- we calculate the d_1 which is distance between intersection point y to the pixel y_k

  So, $d_1 = y-y_k$

  $d_1 = mx_{k+1} +c -y_k$ By using ....(1)

  Similarly d_2 is the distance between pixel y_{k+1} and intersection point y.

  $d_2 = y_{k+1}-y$

  $d_2 = y_{k+1}-(mx_{k+1}+c)$ By using ....(1)

  Note: here x is always incrementing so we can write $\mathbf{x_{k+1}}$ **as** $\mathbf{x_k}$ **+1** and here $\mathbf{y_{k+1}}$ is next pixel so we can write it as y_k+1.
- subtracting d_2 from d_1

  $d_1-d_2 = m(x_k+1) +c -y_k - [y_k+1-(mx_k+1+c)]$

  $= m(x_k+1)+c -y_k - y_k-1+m(x_k+1)+c$

  $d_1-d_2 = 2m(x_k+1)-2y_k+2c-1$ .....(3)

Derivation

$$d_1 - d_2 = 2\frac{dy}{dx}(x_k + 1) - 2y_k + 2c - 1 \qquad \text{by using...(2)}$$

- Multiplying both side by (dx)
  $dx(d_1-d_2) = 2dy(x_k+1)-2dx(y_k)+2dx(c)-dx$
- Now we need to find decision parameter P_K
  $P_K = dx(d_1-d_2)$ and,
- $C = 2dy+2dx(c)-dx$ **which is constant**
- So new equation is.
  $P_K = 2dy(x_k)-2dx(y_k) + C$ .....(4)
- Now our next parameter will be
  $P_{K+1} = 2dy(x_{k+1})-2dx(y_{k+1}) + C$ .....(5)

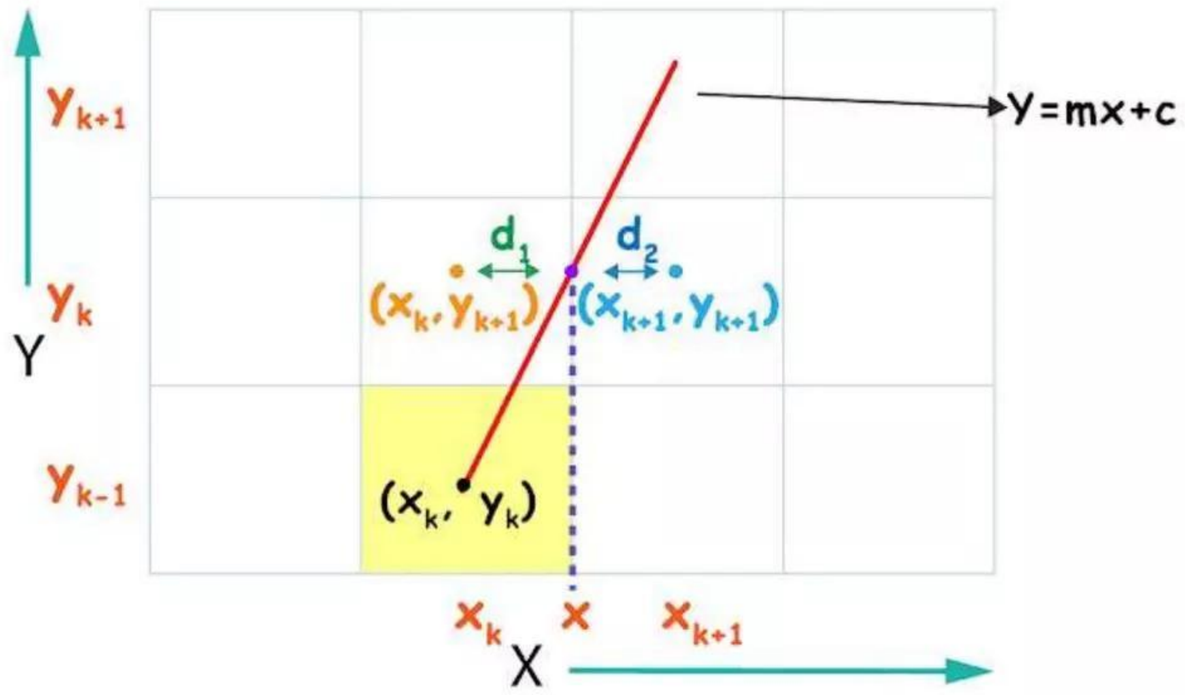
- Subtracting P_k from PK_{+1}
  $P_{k+1}-P_k = 2dy(x_{k+1}-x_k)-2dx(y_{k+1}-y_k) + C-C$
  Note: here x is always incrementing so we can write x_{k+1} as $x_k + 1$
  $P_{k+1}-P_k = 2dy(x_k-x_k+1)-2dx(y_{k+1}-y_k)$
  $P_{k+1} = P_k+2dy-2dx(y_{k+1}-y_k)$.....(6)

- when $P_k<0$ then $(d_1-d_2)<0$
- So $d_1<d_2$ then we will write y_{k+1} as y_k because current pixel's distance from intersection point y is smaller.so, we will have to choose current pixel.
  Then our formula will be:
- $P_{k+1} = P_k+2dy-2dx(y_k-y_k)$
- $P_{k+1} = P_k+2dy$

Derivation

- And when $P_k>0$ then $(d_1-d_2)>0$
- So $d_1>d_2$ then we will write y_{k+1} as y_k+1 because current pixel's distance from intersection point y is larger.so, we will have to choose upper pixel.
- At that time our formula will be:
  $P_{k+1} = P_k+2dy-2dx(y_k+1-y_k)$
  $P_{k+1} = P_k+2dy-2dx$
- We can say that $(y_{k+1}-y_k)$ value can either be 0 or 1.
  For Initial decision parameter
- From 4th equation
  $P_K = 2dy(x_k)-2dx(y_k) +C$
  $P_K = 2dy(x_k)-2dx(y_k)+2dx(c)-dx+2dy$
- By using 1st equation
  $y_k=m(x_k)+c$
  $c=y_k-m(x_k)$
  $P_0 = 2dy(x_k)-2dx(y_k)+2dx(y_k-m(x_k)-dx$ (By using 2)
  $= 2dy(x_k)-2dx(y_k)+2dxy_k-2dyx_k+2dy-dx$
  $P_0 =2dy-dx$

Derivation



- But if the slope of line is greater then 1 (m>1).
  then our Y coordinate will always be incremented and we have to choose between x_k or x_{k+1}.
  So, our Line equation will be:

  $$Y_{k+1} = m(x)+c$$

- Now, In this case our d_1 will be the distance between intersection point x and pixel x_k
  $d_1 = x - x_k$ By using ....(7)
  $d_1 = 1/m(y_{k+1} - c) - x_k$
  And similarly our d_2 will be the distance between intersection point x and

Derivation

- pixel x_{k+1}
  $d_2 = x_{k+1} - x$ By using ....(7)
  $d_2 = x_{k+1} - 1/m(y_{k+1} - c)$
- Note: here y is always incrementing so we can write y_{k+1} as $y_k + 1$ and here x_{k+1} is next pixel so we can write it as $x_k + 1$.
  subtracting d_2 from d_1
  $d_1 - d_2 = 1/m(y_k + 1 - c) - x_k - [x_k + 1 - 1/m(y_k + 1 - c)]$
  $= 1/m(y_k + 1 - c) - x_k - x_k - 1 + 1/m(y_k + 1 - c)$
  $d_1 - d_2 = 2/m(y_k + 1 - c) - 2x_k - 1$
  $d_1 - d_2 = 2dx(y_k + 1 - c) - 2dyx_k - dy$

- Multiplying both side by (dy)
  $dy(d_1 - d_2) = 2dxy_k + 2dx - 2dxc - 2dyx_k - dy$

- Now we need to find decision parameter P_K
  $P_K = dy(d_1 - d_2)$
  $C = 2dx - 2dxc - dy$ which is constant
  So new equation is
  $P_K = 2dxy_k - 2dyx_k + C$ .....(8)

Derivation

- Now our next parameter will be
  $P_{k+1} = 2dx(y_{k+1})-2dy(x_{k+1})+C$

- Substracting P_k from P_{k+1}
  $P_{k+1}-P_k = 2dx(y_{k+1}-y_k)-2dy(x_{k+1}-x_k)+C-C$
  Note: here x is always incrementing so we can write y_{k+1} as $y_k + 1$

  $P_{k+1}-P_k = 2dx(y_k +1-y_k)-2dy(x_{k+1}-x_k)$
  $P_{k+1}=P_k+2dx-2dy(x_{k+1}-x_k)$
  when $P_k<0$ then $(d_1-d_2)<0$
- So $d_1<d_2$ then we will write x_{k+1} as x_k because current pixel's distance from intersection point x is smaller.so, we will have to choose current pixel.

  Then our formula will be:

  $P_{k+1} = P_k+2dx-2dy(x_k-x_k)$
  $P_{k+1} = P_k+2dx$
  And when $P_k>0$ then $(d_1-d_2)>0$
- So $d_1>d_2$ then we will write x_{k+1} as x_k+1 because current pixel's distance from intersection point x is Larger.so, we will have to choose next pixel.
- At that time our formula will be:
  $P_{k+1}= P_k+2dx-2dy(x_k+1-x_k)$
  $P_{k+1}= P_k+2dx-2dy$

Derivation

- For Initial decision parameter
  From 8^{th} equation
  $P_K = 2dxy_k - 2dyx_k + C$
  $P_0 = 2dxy_0 + 2dx - 2dxc - 2dyx_0 - dy$
  By using 1^{st} equation
  $y_k = m(x_k) + c$
- $c = y_k - m(x_k) P_0 = 2dxy_0 - 2dyx_0 + 2dx - 2dx(y_0 - m(x_0)) - dy$
  $P_0 = 2dxy_0 - 2dyx_0 + 2dx - 2dxy_0 + 2dyx_0 - dy$ (By using 2)
  $P_0 = 2dx - dy$

hence formulas for bresenham is derived.

Bresenham's Line Drawing Algorithm

Bresenham's Line Drawing Algorithm

```
#include<stdio.h>
#include<conio.h>
#include<graphics.h>


void main()
{
int gd =DETECT,gm;
initgraph(&gd,&gm,"C:\\TurboC3\\BGI");
int x,x1,y,y1,dx,dy,pk;
printf("Enter first co-ordinates");
scanf(" %d%d",&x,&y);
printf("Enter second co-ordinates");
scanf(" %d%d",&x1,&y1);
dx=x1-x;
dy=y1-y;
pk=2*dy-dx;
```

Bresenham's Line Drawing Algorithm

```
while(x<x1)
      { x++;
    if(pk<0){
          pk+=2*dy;
           }
    else{y++;
    pk+=2*(dy-dx);
         }
     putpixel(x,y,9);
      }
getch();
closegraph();


}
```

Bresenham's Line Drawing Algorithm:Example

- Example on Bresenham's algorithm: Consider the line from (0, 0) to (−8,−4), use general
- Bresenham's line algorithm to rasterize this line. Evaluate and tabulate all the steps involved.
- Solution:
- Given data,
- (x 1, y1)= (0, 0) (x2, y2)= (−8,−4)
- $\Delta x = x2 - x1 = -8 - 0 = 8$
- $\therefore S1 = -1$
- $\Delta y = y2 - y1$
- $= -4 - 0 = 4$
- $\therefore S2 = -1$
- Decision Variable $= e = 2*(\Delta y) - (\Delta x)$
- $\therefore e = 2*(4) - (8)$
- $= 8 - 8 = 0$

Bresenham's Line Drawing Algorithm:Example

- The result in tabulated form as,

Pixel	e	x	y
Initially (0,0)	0	0	0
(-1,0)	+8	-1	0
(-2,-1)	0	-2	-1
(-3,-1)	-8	-3	-1
(-4,-2)	0	-4	-2
(-5,-2)	+8	-5	-2
(-6,-3)	0	-6	-3
(-7,-3)	+8	-7	-3
(-8,-4)	0	-8	-4

Bresenham's Line Drawing Algorithm:Example

- ∴ By pictorial presentation in graph is as shown below,